

GunnerAgent: An Agent Submitted to the ANAC 2026 HAN League

Omer Shani Steinmetz
The College of Management Academic Studies

June 7, 2026

Abstract

GunnerAgent is a bilateral negotiation agent built for the Human-Agent Negotiation (HAN) League of ANAC 2026. The agent is split into two layers: a numerical core that makes every bidding and acceptance decision using a two-phase concession curve and an adaptive opponent model, and an LLM wrapper that generates the natural-language message accompanying every move. The split keeps the strategy reproducible and tunable in isolation from the LLM’s stochastic output and ensures that an LLM failure can never produce a bad numerical move.

1 Introduction

The Automated Negotiating Agent Competition (ANAC) [1] is an international tournament running since 2010. In 2026 it introduced the Human-Agent Negotiation (HAN) League [2], in which the agent negotiates against a human partner using the Alternating Offers Protocol augmented with free-text messages. Sessions are short and human-paced - a five-minute wall clock yields around ten propose calls per side - and the agent is scored both by the utility it achieves and by the partner’s post-negotiation perception of the agent. The protocol therefore couples a tight numerical-strategy budget with a human-facing presentation requirement. Consequently, all underlying functions, definitions, and parameters within our framework are inherently driven by the context of negotiating with a human; in addition to executing a sound competitive strategy, ensuring that the agent’s communication appears authentic and reliable is paramount for establishing credibility.

GunnerAgent meets these two requirements with two clearly separated layers:

- **Numerical core (GunnerCore).** A `NegMAS SAONegotiator` that decides whether to accept the partner’s offer and what to propose next. It implements a two-phase concession curve, a Pareto-filtered propose loop, an opponent-utility estimator built from the partner’s offer history, and a no-repeat cap.
- **LLM wrapper (GunnerAgent).** An `LLMMetaNegotiator` that delegates every numerical decision to the core and only generates the natural-language message that goes out with that decision. The LLM never changes the offer or the accept/reject choice.

This split is what makes the agent reliable: an LLM timeout, a malformed JSON response, or a stochastic-sampling quirk cannot produce a bad numerical move, and the strategy can be tuned independently of the message style. The numerical core takes the aspiration formula from the Shochan agent [3] (ANAC 2024 ANL winner) as a starting point and extends it with the curve and opponent model described below. Section 2 describes the design. Section 3 discusses limitations and possible improvements.

2 The Design of GunnerAgent

2.1 Architecture

GunnerCore, the numerical brain, exposes the standard NegMAS interface (`propose`, `respond`, preference setup). The wrapper forwards every `propose` and `respond` call to the core, takes the returned numerical decision, asks the LLM to produce a message contextualised on that decision, and emits the pair (decision + message) to the mechanism. The wrapper never re-evaluates the decision or generates an offer of its own.

One asymmetry in HAN is particularly relevant to the core’s design. The agent has full access to its own utility function, but the *partner’s utility function is not exposed*. In ANL 2024 the agent is handed the opponent’s utility function directly - the Shochan code calls `opponent_ufun(outcome)` in several places (Pareto computation, candidate sorting, partner reserved-value estimation). HAN does not expose this access. Every such call has to be replaced with an estimate built from the partner’s offer history, and with around ten partner offers per game the estimator has very little data to work with. The opponent model (Section 2.4) is the component that fills this gap.

2.2 Propose / respond loop

On each round, the core uses the same threshold curve in two places:

- **Acceptance.** The partner’s offer is accepted if its utility to us is at or above the current threshold; otherwise it is rejected and we counter-propose in the same turn.
- **Proposal.** The candidate set is the outcomes above the threshold. We filter that set to its Pareto frontier on (my_util, opp_util) , sort by predicted opponent utility in descending order, and propose the highest-ranked outcome that has not already been offered too many times. The no-repeat cap prevents the agent from repeating itself, which a human reads as stonewalling.

The agent therefore only ever proposes outcomes it would itself accept, and within that set it pushes the partner toward what it believes the partner most wants.

2.3 Bidding & acceptance: two phases instead of one

The classical aspiration curve used by Shochan and similar agents is

$$\text{aspiration}(t, M, rv, e) = (M - rv)(1 - t^e) + rv, \quad (1)$$

where t is the relative time, M is the maximum utility, rv is the reservation value, and e shapes the curve.

A single instance of this curve does not survive the HAN budget, regardless of the exponent we pick. A Boulware ($e > 1$) makes the agent re-offer essentially the same outcome round after round, which reads to a human as robotic and not how a person negotiates. A Conceder ($e < 1$) drops the threshold so quickly that we give away most of our value in the first few turns and end the game with very little. A linear curve ($e = 1$) avoids both extremes but is fully predictable: the partner sees the threshold falling at a constant rate and has every incentive to wait until the final turn, when the threshold sits at our reservation value, before agreeing to anything. With only ten turns to express itself, no single exponent does the two jobs the agent actually needs in HAN: signal real movement without giving up real value, and look like a person negotiating rather than a function being evaluated.

GunnerCore splits the game at $t_{\text{split}} = 0.4$ into two phases that play complementary roles:

- **Phase 1 - Anchor.** A linear descent from utility 1.0 to a fixed Phase-1 floor of 0.7. We deliberately do not let this floor scale with rv : an earlier design that scaled it upward kept the threshold near 1.0 throughout Phase 1 on high- rv scenarios, and the agent simply re-offered its maximum-utility outcome every round. With a fixed 0.7, the threshold concedes the same way regardless of rv - the partner sees real movement every round, and we still give up almost no real value. The linear shape is deliberate: over five turns, anything more elaborate is wasted.
- **Phase 2 - Settle.** A Shochan aspiration curve (Equation 1) from 0.7 down to $\max(1.5rv, 0.3)$, with a moderate Boulware exponent ($e = 2.5$). The lower bound of 0.3 keeps the absolute floor meaningful even when rv is small (so we never settle for almost nothing), and the $1.5rv$ term scales the floor up on high- rv scenarios so we never accept a deal that gives us less than $1.5\times$ our walk-away value. The Boulware exponent holds the threshold near 0.7 through most of Phase 2 and concedes sharply only in the final rounds, so the partner sees a clear settling plateau and time pressure does the rest of the work.

2.4 Opponent modelling without an opponent utility

As noted in Section 2.1, GunnerCore cannot call the partner’s utility function. The estimator $\hat{u}_{\text{opp}}(\text{outcome})$ is built from the partner’s offer history alone. It is used only to *rank* candidate outcomes inside the propose loop (Pareto filter and the sort step); it does not influence the acceptance decision.

Why classical frequency models do not transfer. The literature on opponent modelling for AOP (HardHeaded [4], CUHK, Smith, the Fawkes family) is built around frequency tables: “the values the partner offers most often are the ones they prefer.” This is correct when the partner spends hundreds of rounds near their preferred outcomes. It is wrong in HAN. The partner spends only the first few turns near their preferred outcomes and the rest *conceding*, so the frequency table fills with non-preferred values and the inferred ranking degrades round by round. In our tests every frequency model converged to the performance of a zero-sum proxy, which uses no partner information at all.

Template-match against the first offer. What worked was treating the partner’s *first offer* as a preference template. The first offer is the only one we can trust to be near the partner’s preferred outcomes; everything after it is concession. A candidate outcome is then scored by the fraction of issues whose values match the template.

Why one template is not enough. Template-match worked on buyer–seller scenarios, where the first offer reveals the partner’s preferred value on every issue (a seller’s opening of “high price, high quantity” is genuinely informative). It failed on resource-distribution scenarios: there the partner’s first offer is “I get everything” on every issue, which conveys nothing about which issues actually matter to them.

Adaptive split. GunnerCore therefore chooses between two modes at runtime, by inspecting the number of issues in the outcome space:

- **Two issues or fewer (buyer–seller-like).** Plain template-match against the first offer. The first offer is informative enough.
- **Three or more issues (resource distribution).** Template-match *weighted* by per-issue importance learned from the *stability* of the partner’s offers across consecutive rounds. Issues the partner keeps unchanged round after round gain weight; issues they swap drop

in relative weight. The candidate score becomes a weighted sum of issue-matches against the template.

The split needs no scenario metadata - it depends only on the number of issues, which the agent always knows.

We considered also estimating the partner’s reservation value (the Shochan code uses it explicitly) but did not: the budget of partner offers is too small to fit a curve reliably, and a wrong *rv* estimate is more damaging than no estimate.

2.5 LLM message layer

The wrapper’s only role is presentation. Once GunnerCore has fixed the move, the LLM produces the short message that accompanies it; it never selects the offer and never decides whether to accept. The model is `qwen3:4b-instruct`, run locally through Ollama and fixed by the league rules, so only the prompt text and the sampling parameters (temperature, token cap) can be modified.

Modular prompt assembly. The round prompt was first written as a single block of text, which proved difficult to maintain. With all instructions in one block they interfered with one another: changing one instruction, such as how a repeated offer should be phrased, also affected how the model handled the others, and the order in which the model read the instructions could not be controlled. Both effects are significant for a small model. The prompt is therefore assembled from several small Python functions, one per concern: the global rules, the brief for the current move, the reading of the partner’s last action, the offer line, and the output-format instruction. Each component is built and positioned independently, so any one can be revised, reordered, or removed without affecting the others, which gives direct control over the content the model receives and the order in which it appears.

Model inputs and hidden information. Each round the model receives a verbal description of the situation rather than numeric values: a label for the move just made, a summary of the partner’s last action, the exact item names and quantities of the offer, and the partner’s last message. It is never given the agent’s utility values, reservation value, or issue priorities. This prevents the strategy from being reflected in the message and prevents the model from referring to figures it should not disclose.

Move and partner labels. The current move is mapped to one of a small set of labels - opening, large concession, small adjustment, holding firm, repeating, re-raising a previous offer, and accepting - each associated with a fixed tone and a one-line writing brief, so that the message corresponds to the move being made. A separate, lighter classification of the partner’s last action allows the message to respond to the partner as well.

Fallback behaviour. The numerical move is independent of the model. If the model times out, returns malformed JSON, or produces an unusable response, a predefined fallback message for the corresponding behaviour is sent with the move, so the agent is never silent and never sends text that conflicts with its offer.

Prompt-engineering observations. A small model requires explicit, unambiguous instructions. Three observations shaped the final prompt:

- **Recency.** The model gives the most weight to the text it reads last, so the exact offer terms and any essential instruction are placed at the end of the prompt, after the history, rather than at the start.

- **Literal reproduction.** The model tends to copy phrasing from the prompt directly. Rather than allow it to rephrase the offer, which produced incorrect items, it is given a single ready-made “You get ...” sentence to reproduce exactly.
- **Conflicting instructions.** When two instructions address the same point, the model often follows neither reliably, so each point is stated once, in a single place.

Lowering the temperature to 0.7 and increasing the token limit eliminated the remaining paraphrasing and truncation errors.

3 Discussion

GunnerCore is a deliberately compact agent: two phases, one opponent-model switch, and a no-repeat cap. The design rests on the hypothesis - supported by our development experiments - that most of the value extractable in a HAN game comes from *holding firm long enough for the partner’s concession to meet our threshold near the middle of the game*. The opponent model exists to break ties among many acceptable outcomes; the threshold curve is what extracts value.

Two limitations remain. First, when the outcome space is small, a high Phase 2 threshold leaves only a handful of candidates above it, and the no-repeat cap exhausts them within a few rounds; the propose loop then falls back to re-offering one of them, which appears to the human as the agent being stuck. The message layer softens this by warmly acknowledging the deadlock when the partner keeps repeating, but it cannot fully hide that the offer is not moving. Second, the stability-based opponent model assumes the partner concedes on less-important issues first; a partner who deliberately concedes on important issues to mislead us would fool the model. Possible improvements include opponent classification on top of the first offer to adapt the agent’s own concession exponent, in the spirit of the ChargingBoul [5] agent, and direct tracking of the order of issue-by-issue concessions to extract issue importance more sharply than the stability heuristic does.

References

- [1] ANAC 2026 Competition. <https://anac.cs.brown.edu/>.
- [2] Y. Mohammad, R. Aydogan, and R. Hadfi. *ANAC 2026 HAN League: Call for Submissions*. IJCAI 2026 Competition Track, 2026.
- [3] Takamatsu Family. *Shochan: Winning Agent of the ANAC 2024 ANL*. Source: [anl-agents/anl2024/takafam](https://github.com/anl-agents/anl2024/takafam).
- [4] K. Hindriks and D. Tykhonov. *Opponent Modelling in Automated Multi-Issue Negotiation Using Bayesian Learning*. In *Proceedings of AAMAS*, 2008.
- [5] *The ChargingBoul Agent*. ANAC 2022 ANL runner-up. arXiv:2512.06595.